

The present work was submitted to the Chair of Computational Science and Engineering.

presented by / von

Eren Edgü

476194

Projektarbeit

Project Report

Soft-Sensors to Measure Energy Consumption of Topographic Uncertainty Propagation in Mass Flow Simulations

Soft-Sensoren zur Messung des Energieverbrauchs bei der
topografischen Unsicherheitsfortpflanzung in
Massenstromsimulationen

Supervisors / Betreuer:

Alan Correa

Aachen, 03.08.2026

DECLARATION OF OWN WORK

I hereby declare that this thesis has been written independently and that no sources other than those cited have been used. All passages taken verbatim or paraphrased from published or unpublished works have been clearly marked as such. This work has not been submitted, in whole or in part, for any other academic qualification.

DECLARATION OF AI TOOL USAGE

The following AI tools were used during the preparation of this thesis:

- **Gemini (Google)** — Used for grammar checking, text polishing, literature search assistance, and refining LaTeX formatting across all chapters.
- **GitHub Copilot** — Used for code generation, code completion, and script optimization.

All AI-assisted content has been reviewed and verified by the author. No AI tool was used to generate results, draw conclusions, or replace independent scientific reasoning.

Aachen, 03.08.2026

Eren Edgü

CONTENTS

LIST OF FIGURES	VII
LIST OF TABLES	IX
1 INTRODUCTION	1
1.1 Motivation	1
1.2 Research Questions	2
1.3 Outline	3
2 RELATED WORK	5
2.1 Topographic Uncertainty Quantification in Hydrodynamic Modeling	5
2.2 Energy Approaches in Hydrodynamic Modeling	6
2.3 Limitations of Existing Approaches	7
3 METHODOLOGY AND EXPERIMENTAL SETUP	9
3.1 Baseline Environment and Tools	9
3.1.1 Hardware	9
3.1.2 Target Flood Solver	9
3.1.3 Energy Measurement Framework	10
3.2 SynxFlow-Alumet Integration and Experimental Workflow	10
3.2.1 Single-Run Pipeline	11
3.2.2 Ensemble Creation	12
3.2.3 Post-Processing and Ensemble Analysis	13
4 EVALUATION AND EXPERIMENTAL RESULTS	15
4.1 Study 0: Single-Run Characterization and Code-Phase Profiling	15
4.1.1 Results of Study 0	16
4.2 Study 1: Evaluation of the Time-to-Solution Energy Proxy	17
4.2.1 Experimental Setup	18
4.2.2 Statistical Framework and Metrics	18
4.2.3 Results of Study 1	20

Contents

4.3	Study 2: Physical Hazard Uncertainty Quantification	24
4.3.1	Experimental Setup	24
4.3.2	Statistical Framework	25
4.3.3	Results of Study 2	25
4.4	Study 3: Energy Consumption Uncertainty Propagation	27
4.4.1	Experimental Setup	27
4.4.2	Statistical Framework	27
4.4.3	Results of Study 3	28
5	SUMMARY AND FUTURE WORK	33
5.1	Summary	33
5.2	Limitations	34
5.3	Future Work	34
	BIBLIOGRAPHY	35

LIST OF FIGURES

3.1	Single-run execution workflow	11
3.2	Monte Carlo Pipeline Flowchart	13
4.1	Process-attributed CPU energy time series during single-run execution. Generated using <code>alumat-insight</code> [8].	16
4.2	Process-attributed GPU energy time series during single-run execution. Generated using <code>alumat-insight</code> [8].	16
4.3	Cumulative energy timeline (CPU + GPU) for a single simulation run.	17
4.4	Grid comparison of the DEM resampled across four resolutions to alter solver execution workloads.	19
4.5	Total energy consumption vs. execution duration for all iterations.	21
4.6	Total energy consumption vs. execution duration for all clean iterations.	22
4.7	Average cumulative energy-time plot of ensembles with different resolutions.	23
4.8	Boxplots of total energy consumption for each topographic noise level for run 1	28
4.9	Boxplots of total energy consumption for each topographic noise level for run 2	29
4.10	Boxplots of total energy consumption for combined results of runs 1 and 2	30

LIST OF TABLES

4.1	Welch’s t-test results comparing the average power draw between different grid resolution ensembles.	22
4.2	Impact of input uncertainty σ_{input} on the resulting water depth measurements.	26
4.3	Statistical summary of total computational energy consumption for Run 1 (100 iterations).	28
4.4	Statistical summary of total computational energy consumption for Run 2 (100 iterations).	29
4.5	Statistical summary of total computational energy consumption for the combined 200-iteration ensembles.	31

1 INTRODUCTION

Natural hazards like landslides and river floods frequently cause deaths, economic devastation, and environmental destruction worldwide [14, 25]. To assess the risks of these hazards, geohazard studies rely on computational flood models [14, 25]. An essential component in developing any of these computational models is the Digital Elevation Model (DEM). DEMs represent the topography of the terrain where the landslide or flood occurs [25].

Despite DEMs being essential components, these simulations are susceptible to vertical inaccuracies in the topographic data [14]. However, in typical simulation-based hazard assessments, the effect of the DEM error is rarely considered in uncertainty propagation [25].

Digital Elevation Models (DEMs) derived from spaceborne sensors, such as Synthetic Aperture Radar (SAR) or stereoscopic imaging, inherently exhibit vertical inaccuracies due to varying penetration of vegetation and built infrastructure [14]. This frequently introduces a positive elevation bias, overestimating ground heights. In hydrodynamic simulations, these artificial errors act as synthetic sinks or obstructions that mistakenly retain or divert flow, ultimately compromising hazard maps [14].

Stochastic simulation is an efficient method to quantify uncertainty [25]. By treating the DEM error as a random field and injecting it into the original data, an ensemble of equiprobable DEM realizations can be generated [15, 25]. As a result, when comparing an original DEM with a synthetically noisy DEM, both must be regarded as alternative, equally probable representations of the actual landscape [15].

1.1 MOTIVATION

In flood simulation UQ (Uncertainty Quantification), hundreds or thousands of synthetic scenarios must be evaluated to generate the required ensembles [2, 10, 25]. Because high-accuracy numerical flood models are already computationally expensive, this scale of analysis creates a massive computing demand that is beyond the

capabilities of conventional supercomputers [2, 10]. To reduce the power and operating costs associated with these high-resolution simulations, the flood modeling community is searching for hardware solutions that offer better performance-per-watt, such as multi-GPU architectures.

Nevertheless, execution time is still implicitly used as a proxy for computational efficiency [1, 13, 18]. This ignores actual energy usage, despite the shift to power-hungry multi-GPU architectures. Instead of monitoring electrical power draw, researchers tend to use runtime benchmarks and speedups to assess the performance of new GPU-accelerated solvers [1, 13, 18].

For instance, recent hardware implementations achieves up to a 36.6-fold speedup over CPU baselines, reducing high-resolution global simulation runtimes from days to hours [13]. The continuous power needed by the several A100 GPUs to reach this performance is not measured by the authors. In similar fashion, the developers of the LISFLOOD-FP 8.0 model evaluate their new GPU solvers only on the basis of "relative runtime cost," characterizing the solvers as highly efficient mainly due to their ability to execute large-scale catchment simulations "faster than real-time." [18]

This reliance on the time-proxy extends across the broader geohazard domain. In landslide runout modeling, the transition to GPU architecture in the ORIN-3D model is praised for providing an "increase in efficiency of over two orders of magnitude" [1]. However, this efficiency is calculated by the fact that the fully optimized code finishes 115 times faster than its CPU counterpart [1].

These papers ignore the power requirements of modern GPUs by evaluating model efficiency just based on wall-clock time and equating speed with efficiency [1, 13, 18]. This leaves a gap in our understanding of the true energy footprint required to run large-scale uncertainty quantification ensembles.

1.2 RESEARCH QUESTIONS

To address the research gap identified above, this study derives the following primary research question:

- How do input data uncertainty and execution environment volatility impact the measurement and predictability of computational energy consumption in GPU-accelerated flood solvers?

To comprehensively answer this problem, the research is guided by three specific sub-questions:

- What methodologies enable accurate process-level energy attribution in shared heterogeneous computing infrastructures?
- To what extent can execution time serve as a proxy for energy consumption in heterogeneous CPU-GPU computing systems?
- How does uncertainty in simulation inputs propagate to variability in computational energy consumption?

1.3 OUTLINE

The remainder of this report is structured as follows. In Chapter 2, we review existing literature on topographic uncertainty quantification and computational energy profiling, highlighting the limitations of current telemetry methodologies. In Chapter 3, we detail the experimental setup, introducing the SynxFlow solver, the Alumet agent, and the custom orchestration framework developed for automated ensemble execution and data synchronization. In Chapter 4, we present the empirical findings across four studies, which validate the process-isolated telemetry approach, evaluate execution time as an energy proxy, and assess how topographic noise impacts both physical flood predictions and computational energy variability. Finally, in Chapter 5, we summarize the core findings, acknowledge the study's limitations, and propose directions for future research.

2 RELATED WORK

This chapter reviews existing literature at the intersection of hydrodynamic flood modeling, digital terrain analysis, and energy telemetry in high-performance computing (HPC). Section 2.2 examines current software and hardware profiling methodologies used in hydrological simulations. Subsequently, Section 2.3 analyzes the key limitations of component and node level measurement techniques, highlighting the need for process-isolated telemetry.

2.1 TOPOGRAPHIC UNCERTAINTY QUANTIFICATION IN HYDRODYNAMIC MODELING

Topographic UQ is a well established discipline in computational hydraulics, evaluating how vertical elevation errors in DEMs propagate through two-dimensional Shallow Water Equations (SWE) [3, 12, 17]. Forward UQ frameworks routinely use non-intrusive Monte Carlo (MC) sampling to stochastically perturb baseline DEMs and derive probabilistic confidence bounds [3, 17]. While Multilevel Monte Carlo (MLMC) sampling has been established to accelerate shallow-water UQ under structural parameter and fault uncertainties [16], spatial DEM elevation noise propagation in 2D SWE solvers has predominantly relied on standard MC sampling. Across these frameworks, existing literature has focused almost exclusively on domain-level physical metrics, quantifying output variance in localized water depths, flow velocities, and total flood inundation boundaries [3, 12, 16, 17].

While physical output variability is thoroughly documented, the associated computational burden is traditionally reported strictly as static wall-clock time or CPU execution penalties [3, 12, 16, 17]. In explicit numerical schemes, localized elevation perturbations directly alter local water depths h , causing inter-cell fluxes, Courant-Friedrichs-Levy (CFL) stability constraints, and allowable time steps (Δt) to fluctuate dynamically across realizations [3, 12, 16, 17]. Consequently, a gap exists between uncertainty propagation research and hardware energy telemetry in computational

hydraulics. Within the examined UQ literature for 2D shallow water modeling, real-time hardware power draw and energy metrics across stochastic DEM ensembles remain unexamined. Investigating energy consumption in topographic UQ ensembles directly addresses this gap, establishing whether single-run deterministic profiling can accurately predict compute allocations for stochastic workloads on shared HPC infrastructure.

2.2 ENERGY APPROACHES IN HYDRODYNAMIC MODELING

While traditional HPC research in hydrology often focuses on minimizing execution time, recent literature has shifted toward profiling the energy efficiency of flood simulations and terrain analyses. To achieve this, researchers have employed a variety of software and hardware telemetry methods to capture the power consumption of hydrodynamic models.

To address the challenge of achieving performance portability for a 2D shallow water equation solver across diverse heterogeneous hardware (CPUs, GPUs, and FPGAs) without maintaining entirely separate code bases, researchers recently evaluated a unified SYCL implementation [7]. To measure energy across diverse hardware, the authors used the Power Measurement Toolkit (PMT) to unify different measurement APIs into a single interface [7]. PMT creates a background worker thread that periodically polls native hardware energy counters to calculate the total energy consumed over the entire simulation loop [7]. To ensure fairness, they established strict hardware boundaries by measuring only the energy of the active accelerator device while explicitly neglecting the energy of the host CPU controlling the simulation [7].

Another study evaluated the performance, usability, and energy efficiency of using high-level Python wrappers compared to native C++ for accelerating computationally bound shallow-water equation models on GPUs [11]. To measure energy, the authors employed two methods depending on the hardware [11]. For high-end server GPUs, they ran the `nvidia-smi` utility in the background to log power draw profiles of the GPU every 20 milliseconds, while for desktop and laptop systems, they used a physical watt meter to measure the total power consumed by the entire computer [11]. In both methods, they recorded the idle power draw before and after the benchmarks and subtracted it from the final measurements to isolate the simulation's active energy footprint [11].

In the broader domain of digital terrain analysis, researchers aimed to accelerate the computationally and data-intensive Multiple Flow Direction (MFD) algorithm by parallelizing it across multi-core CPUs and many-core GPUs [20]. To evaluate the energy efficiency of their CUDA and OpenACC implementations, the authors used the MeterPU software library [20]. This library allowed them to directly capture and compare the total energy consumption of both the CPU and the GPU architectures during the execution of the algorithms [20].

Finally, a recent study investigated energy optimization strategies when using HPC to forecast floods. The researchers tested different computer cluster setups to find the most efficient way to run TELEMAC-2D, which is a program used to simulate flash floods in the Brague river [4]. To profile energy consumption, they used a Python script based on the ska-sdp-perfmon toolkit to collect power data directly from the computer nodes [4]. By tracking this power usage over time, they calculated the total energy needed to complete a simulation. They noted that because they measured energy at the node level, the final numbers included both the flood simulation itself and any background programs running on those same nodes at the time [4].

2.3 LIMITATIONS OF EXISTING APPROACHES

Despite providing valuable macro-level insights, these software and hardware telemetry methodologies exhibit limitations that compromise their precision. A fundamental challenge is that modern hardware frequently executes multiple concurrent tasks. However, native hardware sensors report the total power consumption of the component rather than isolating the energy consumed by a specific process. Consequently, measuring GPU or node energy also captures background activities not initiated by the simulation itself. While baseline subtraction techniques can successfully identify broad efficiency trends, they can also yield misleading results due to the unpredictable nature of operating system tasks. For instance, if a background spike occurs during the baseline check, the resulting baseline can be skewed.

Beyond hardware-level measurement challenges, software-based profiling frameworks present distinct operational constraints. Many of these tools rely on white-box methodologies that require manual modification to the solver's source code by inserting explicit instrumentation hooks or measurement calls. Furthermore, these telemetry libraries are typically language-specific which is restricting their portability.

3

METHODOLOGY AND EXPERIMENTAL SETUP

This chapter details the methodology and software framework developed to couple hydrodynamic flood modeling with process-isolated energy telemetry. First, Section 3.1 describes the baseline computational environment, introducing the SynxFlow hydrodynamic solver, and the Alumet telemetry agent. Section 3.2 presents the custom Python orchestration framework, detailing the single-run staging pipeline, the Monte Carlo ensemble execution, and the post-processing data synchronization methods.

3.1 BASELINE ENVIRONMENT AND TOOLS

3.1.1 HARDWARE

The numerical simulations were executed on the MBD high-performance compute server Gaia. Gaia cluster is equipped with 96 CPU cores, 1 TB of RAM, and 4 dedicated GPUs, providing the parallel processing capacity required to handle computationally intensive simulation workloads.

3.1.2 TARGET FLOOD SOLVER

SynxFlow is an open-source Python package designed to simulate multi-hazard events, including flood inundation, landslide runouts, and debris flows [23]. The flood solver relies on 2D shallow-water equations solved via Godunov-type shock-capturing finite volume methods [22, 23]. Crucially, SynxFlow addresses long-standing stability issues in shallow-water modeling through a Surface Reconstruction Method and an implicit friction discretization scheme [22]. These improvements ensure the accurate calculation of slope source terms and maintain numerical stability even in the limit of disappearing water depths or across highly irregular topographies with abrupt slope changes [22]. The software is divided into two distinct components:

a high-performance C++/CUDA backend containing the physical solvers, and a Python Input/Output frontend that natively interfaces with standard libraries such as NumPy, Pandas, and Rasterio [23, 24].

SynxFlow was chosen as the target hydrodynamic simulation framework for this study due to several advantages that align with the evaluation of energy telemetry. First, SynxFlow utilizes a hybrid execution model where both the CPU and GPU are active. The shallow-water equations are offloaded to CUDA-enabled GPUs, while the host CPU executes the Python IO interface to process data, manage boundaries, and orchestrate the simulation [23]. This creates a heterogeneous and concurrent execution profile, making it a suitable candidate for evaluating process-level telemetry using Alumet. Furthermore, because this orchestration is entirely script-based rather than relying on traditional, low-level configuration files, it enables extensive workflow automation [23].

3.1.3 ENERGY MEASUREMENT FRAMEWORK

To overcome the challenges mentioned in Section 2.3, this study utilizes Alumet, a lightweight and modular framework designed for high-precision local and distributed measurements [5]. Alumet addresses the limitations of traditional hardware sensors by operating a measurement pipeline that simultaneously gathers data from both low-level hardware components (such as Intel RAPL for CPUs or NVML for NVIDIA GPUs) and operating system interfaces (such as procfs or cgroups for software metrics) [5].

Rather than relying only on power readings that also capture background cluster noise, Alumet leverages the energy-attribution plugin [5]. This plugin combines the total hardware energy consumption data with the process-specific hardware utilization metrics [5]. By scaling the total component energy by the specific CPU or GPU utilization percentage of a target process, the framework calculates the energy consumed for the isolated process.

3.2 SYNxFLOW-ALUMET INTEGRATION AND EXPERIMENTAL WORKFLOW

SynxFlow and Alumet operate as separate software tools without interoperability. To bridge this gap, a custom Python-based framework was developed to combine the two tools into an automated experimental pipeline [9].

The workflow is structured around a configurable Monte Carlo loop. In each iteration, a distinct Gaussian noise perturbation is applied to the original DEM. Then the Synxflow solver is executed and energy profiling is done using Alumet, before accumulated telemetry and flood solver data across the ensemble are post-processed.

3.2.1 SINGLE-RUN PIPELINE

The single-run pipeline in Figure 3.1 represents the foundational layer of the framework. For each simulation run, the process begins by preparing a unique topographical landscape.

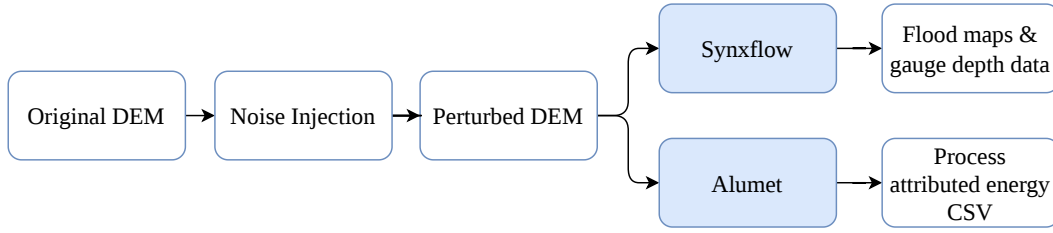


Figure 3.1: Single-run execution workflow

Based on the standard deviation (σ) specified in the configuration file, Gaussian statistical uncertainty is injected into the baseline DEM. For every individual grid cell at spatial coordinates (x, y) , the perturbed elevation $Z_{\text{perturbed}}(x, y)$ is calculated as:

$$Z_{\text{perturbed}}(x, y) = Z_{\text{original}}(x, y) + \epsilon, \quad \text{where } \epsilon \sim \mathcal{N}(0, \sigma^2) \quad (3.1)$$

where $Z_{\text{original}}(x, y)$ represents the baseline elevation and ϵ is a zero-mean ($\mu = 0$) normally distributed random variable. To simplify the error model, the elevation noise is assumed to be spatially stationary and independent of both the absolute elevation (Z) and the local terrain gradient. Furthermore, the surveying equipment is assumed to introduce no directional or systematic spatial bias. Consequently, this produces a perturbed terrain raster that models isotropic measurement errors without introducing artificial elevation drift across the domain.

Once the perturbed DEM is generated, the orchestration script configures the physical environment for the flood simulation. The framework loads the perturbed DEM and establishes the spatial limits of the domain. It configures an open up-stream boundary to inject a dynamic, time-varying water discharge (initiating at

1000, m³/s and decreasing over the simulation period), alongside an open downstream boundary designed to maintain a constant water depth of 5 meters. With the help of Synxflow API, the environment can be customized based on the scenario's specific requirements, allowing the model to simulate continuous rainfall or apply spatially varying surface friction to represent different landcover types.

With the physical domain established, the orchestration framework finalizes the starting conditions by assigning initial water depths (h_0) and placing a spatial observation gauge to track water height over time. Once all environmental parameters are compiled, the simulation is dispatched. During this phase, Synxflow, as mentioned before, utilizes a heterogeneous hardware model. The host CPU orchestrates the data pipeline and input/output operations, while the computationally intensive 2D shallow-water equations are offloaded to CUDA-enabled GPUs for parallel processing.

Simultaneously, the Alumet framework operates as a background daemon to continuously capture the hardware's energy footprint. Dictated by the Alumet configuration file, the agent polls host operating system metrics and CPU registers every 50 ms to measure processor utilization and power draw. Concurrently, it polls the NVIDIA GPUs every 200 ms to log board-level power consumption and Streaming Multiprocessor utilization. The 50 ms CPU polling interval provides high temporal resolution to capture rapid power fluctuations without generating excessive log volume, whereas 200 ms was selected for GPU telemetry to stay within NVML query limits.

Crucially, rather than recording raw, unattributed node-level energy that includes background system noise, the framework employs a mathematical attribution model to isolate the simulation's exact footprint. For instance, the process-attributed GPU energy is dynamically computed at each time step as:

$$\text{Energy}_{\text{attributed, GPU}} = \text{Energy}_{\text{GPU, total}} \times \text{Usage}_{\text{GPU process}} \quad (3.2)$$

This concurrent execution methodology ensures that the completion of every single run yields both the flood solver's results and process-isolated hardware energy profile.

3.2.2 ENSEMBLE CREATION

While the single-run pipeline defines the core execution of an individual simulation, evaluating the impact of topographical uncertainty requires scaling this pro-

cess into an ensemble. To achieve this, an orchestration framework automates the single-iteration pipeline across a configurable Monte Carlo ensemble, as illustrated in Figure 3.2.

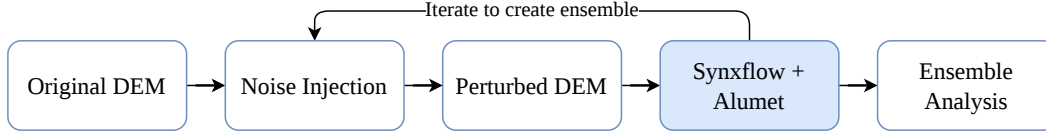


Figure 3.2: Monte Carlo Pipeline Flowchart

To quantify uncertainty across the ensemble, the orchestrator establishes a target observation point to continuously record localized water depth time series. Before running the ensemble loop, a baseline map is generated to visually confirm the monitoring point locations.

To connect SynxFlow and Alumet, the orchestrator dispatches an execution loop that sequentially repeats the single-run pipeline, generating new terrain noise, running the GPU-accelerated solver, and capturing background energy telemetry for every iteration $i \in \{1, \dots, N\}$. The end result is a multi-run dataset combining Synxflow outputs with process-isolated energy profiles, ready for ensemble post-processing.

3.2.3 POST-PROCESSING AND ENSEMBLE ANALYSIS

Following the completion of the automated ensemble loop, the post-processing script synthesizes the raw, asynchronous outputs from both SynxFlow and Alumet into synchronized statistical metrics and visual analyses.

To isolate SynxFlow’s exact energy footprint from background operating system noise, the script performs dynamic Process ID (PID) tracking. Because Alumet logs performance metrics for the entire compute node, execution logs are parsed to identify the specific PID assigned to the SynxFlow solver during each iteration. Filtering the raw Alumet telemetry by this PID extracts only the process-attributed energy.

To resolve the sampling rate, where Alumet polls CPU metrics every 50 ms while GPU metrics update every 200 ms, the script performs timeline synchronization. It constructs a master chronological timeline and linearly interpolates both energy streams onto this unified axis. Linear interpolation was selected over standard polynomial splines because it guarantees monotonicity in cumulative energy and avoids

3 Methodology and Experimental Setup

artificial overshooting between discrete sampling points. This ensures that cumulative CPU and GPU energy footprints are aligned with each other.

In parallel with hardware profiling, the script evaluates hazard uncertainty. The maximum water depth ($h_{\max}$) at designated observation points is extracted for each ensemble iteration. From these datasets, the script derives probability density distributions of peak water depths, quantifying how initial topographical noise translates into uncertainty.

Finally, the script computes ensemble statistics and generates analytical plots for cumulative energy over time, per-iteration energy costs, and energy distributions.

4 EVALUATION AND EXPERIMENTAL RESULTS

4.1 STUDY 0: SINGLE-RUN CHARACTERIZATION AND CODE-PHASE PROFILING

Before scaling the framework up to execute full UQ ensembles, it is necessary to validate Alumet’s ability to accurately isolate and profile the energy consumption of a single simulation run. Because SynxFlow relies on a heterogeneous execution model, its energy draw varies dynamically between the CPU and the GPU depending on the active computational phase.

By correlating the time-series telemetry gathered by Alumet with the cumulative energy curve, the execution pipeline of a single run can be decomposed into two distinct operational phases.

PHASE 1: CPU-BOUND INITIALIZATION AND I/O (0.0 s – 1.0 s)

The simulation begins with a setup phase bound entirely to the host CPU. As shown in Figure 4.1, the CPU telemetry exhibits two distinct power spikes within the first second of execution, while the GPU remains idle near zero active process power. Correlating these spikes with the codebase reveals their reasons. The initial spike corresponds to data ingestion and spatial mapping, where configuration files are parsed, the DEM is loaded into memory via the `IO.Raster` module, and boundary condition arrays are defined. The second spike corresponds to file compilation, where executing `case_input.write_input_files()` finalizes environmental matrices and writes the target simulation files to disk.

As reflected on the cumulative energy curve (Figure 4.3), these two I/O operations create a discrete stair step accumulation of approximately 50 J before the solving begins.

4 Evaluation and Experimental Results

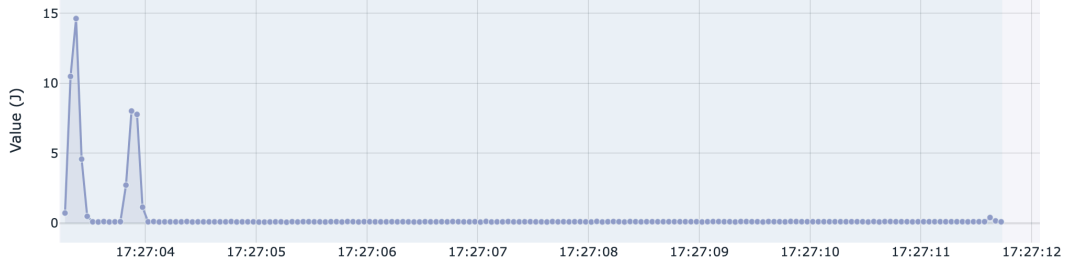


Figure 4.1: Process-attributed CPU energy time series during single-run execution. Generated using `alumet-insight` [8].

PHASE 2: GPU-BOUND SOLVER EXECUTION (3.0 s – 8.5 s)

Following an idle memory transfer window (1.0 s – 2.7 s), the execution paradigm shifts. CPU power consumption drops to near zero as the process waits on CUDA kernel execution. Simultaneously, the GPU telemetry (Figure 4.2) exhibits a sharp power ramp, maintaining sustained power draw throughout the remainder of the iteration.

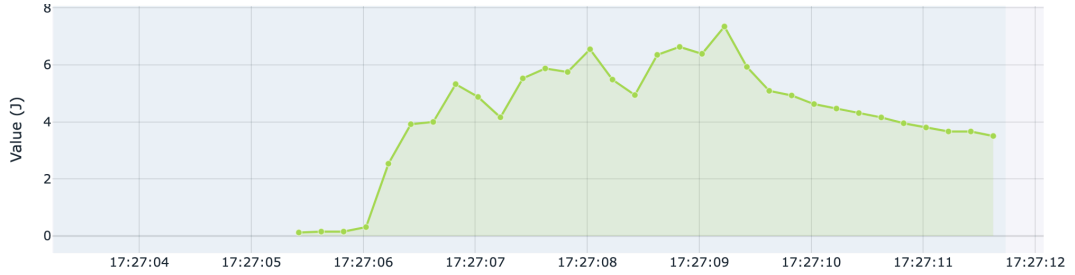


Figure 4.2: Process-attributed GPU energy time series during single-run execution. Generated using `alumet-insight` [8].

This phase corresponds directly to the dispatch of the `flood.run()` solver command, where the 2D shallow-water equations are offloaded to CUDA-enabled GPUs for massively parallel processing. On the cumulative timeline (Figure 4.3), this shows as a constant almost linear slope, driving the total single-run energy cost to approximately 205 J.

4.1.1 RESULTS OF STUDY 0

The observed behavior of the energy curves closely matches the expected execution sequence of the codebase. Therefore these results suggest that the Alumet

4.2 Study 1: Evaluation of the Time-to-Solution Energy Proxy

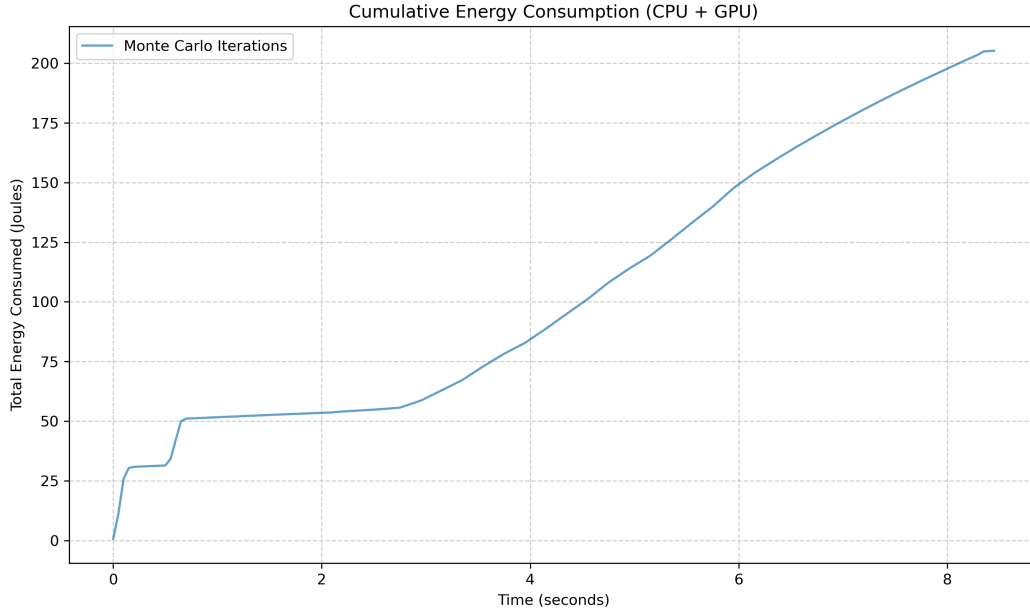


Figure 4.3: Cumulative energy timeline (CPU + GPU) for a single simulation run.

agent, combined with the dynamic PID-tracking methodology, successfully isolates process-attributed energy across heterogeneous hardware.

Demonstrating that the framework can distinguish CPU setup spikes from continuous GPU solver draw provides confidence in the methodology, validating the approach to scale up to the large-scale Monte Carlo ensemble in the subsequent studies.

4.2 STUDY 1: EVALUATION OF THE TIME-TO-SOLUTION ENERGY PROXY

To address the reliance on execution time as a proxy for energy consumption (as highlighted in Section 1.1), this study evaluates whether time-to-solution reliably predicts process-attributed energy draw across varying mesh resolutions. Specifically, this experiment addresses Sub-question 2 by assessing the linearity of the time-energy relationship under deterministic ($\sigma = 0.0$ m) execution conditions.

4.2.1 EXPERIMENTAL SETUP

To systematically vary the computational workload while preserving similar macro-level flood dynamics, the baseline DEM was resampled into three additional spatial resolution tiers: 10.0 m, 12.0 m, 15.0 m, and 20.0 m, as illustrated in Figure 4.4. Adjusting the resolution forces the SynxFlow solver to process different grid dimensions, thereby creating distinct execution durations without altering the underlying physical scenario.

For each DEM resolution, an ensemble of 100 iterations was executed with the topographic noise strictly set to zero ($\sigma = 0.0$ m). Setting the input noise to zero holds the physical elevation raster completely identical across all 100 iterations of a given resolution. In a deterministic computing environment, identical inputs should yield the same execution time and energy consumption. Because the terrain input remains completely constant in this experiment, any observed variability isolates execution uncertainty rather than physical terrain noise.

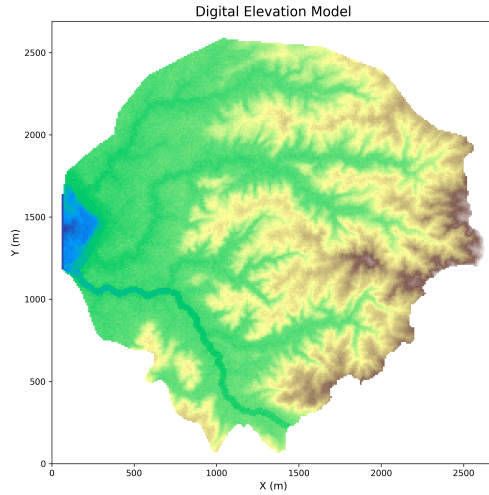
4.2.2 STATISTICAL FRAMEWORK AND METRICS

To accurately reflect the dynamic execution environment and evaluate the time-proxy relationship across the resolution ensembles, the telemetry data is analyzed using the following statistical framework:

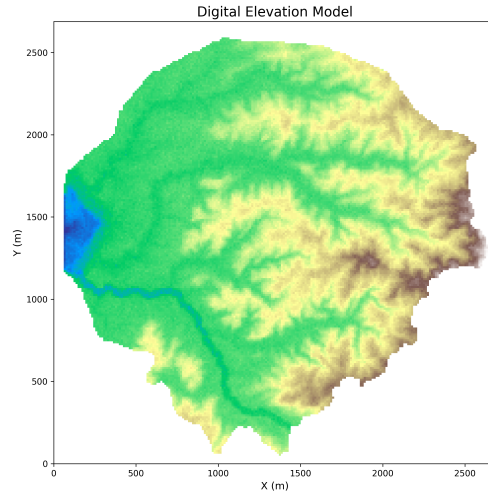
DATA FILTRATION VIA Z-SCORES: In shared High-Performance Computing (HPC) environments, random background operating system spikes or node communication delays can cause execution stalls that do not reflect the solver’s actual mathematical workload. To isolate the solver’s true efficiency, Z-scores are computed for both total execution duration and energy-per-second. Runs exceeding a defined standard deviation threshold (e.g., $Z > 2.0$) are flagged as outliers and removed from the clean dataset. This ensures that the subsequent metrics evaluate the hydrodynamic solver rather than transient cluster traffic.

AVERAGE POWER: For each ensemble, the mean power draw is defined as the slope of a linear regression line fit to the cumulative energy curve. Calculating the mean (μ) of this power draw across the ensemble establishes the true hardware baseline for each spatial resolution tier.

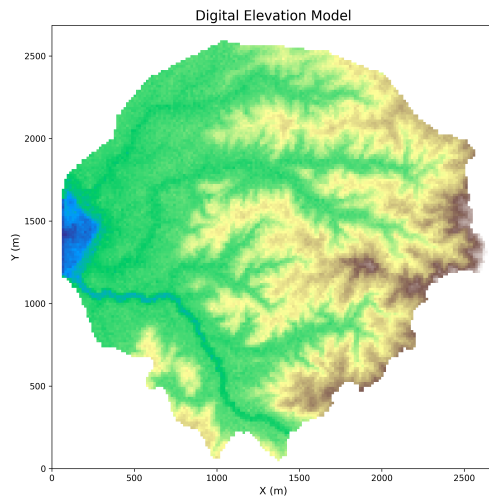
4.2 Study 1: Evaluation of the Time-to-Solution Energy Proxy



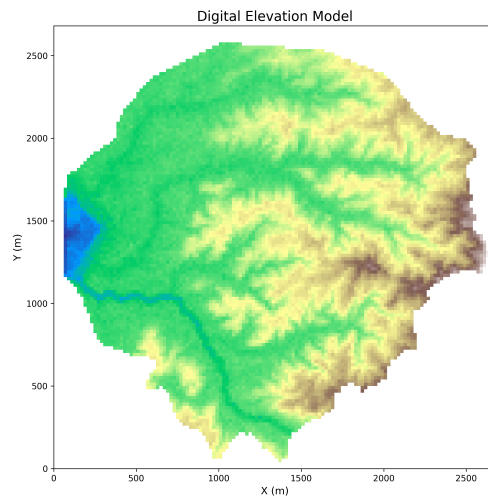
(a) Baseline DEM (10.0 m resolution).



(b) Resampled DEM (12.0 m resolution).



(c) Resampled DEM (15.0 m resolution).



(d) Resampled DEM (20.0 m resolution).

Figure 4.4: Grid comparison of the DEM resampled across four resolutions to alter solver execution workloads.

(R^2) COEFFICIENT OF DETERMINATION: This metric serves as the primary analytical tool to directly test the time-proxy assumption. Fundamental physical power modeling relies on the relationship:

$$E = A + P \cdot t \quad (4.1)$$

where E is total energy, A represents static idle energy overhead, P is dynamic power draw, and t is execution time [21]. If time is a perfect proxy for energy, dynamic power (P) must remain constant across iterations, meaning execution time and total energy would form a perfectly linear relationship ($R^2 = 1.0$). By running a linear regression across the execution times and total energy costs of the iterations, the framework directly tests this linearity. A reduced R^2 value points toward dynamic power fluctuations across runs, which undermines the assumption that execution time can be used as a simple standalone proxy.

WELCH’S T-TEST (INDEPENDENT SAMPLES): To determine how varying computational workloads affect underlying hardware behavior, Welch’s t-test is applied to compare the average power draw between the different DEM resolution groups. This test determines whether altering the grid dimension causes a statistically significant shift in the rate at which the hardware consumes energy.

4.2.3 RESULTS OF STUDY 1

The telemetry data extracted from the ensembles provides three key observations regarding the validity of using execution time as an energy proxy.

The raw telemetry data highlights potential cluster volatility before statistical filtering is applied. As shown in Figure 4.5, tracking the total energy against execution duration for all iterations reveals instances of what appear to be execution stalls. While several outlier runs experienced extended durations, jumping from a baseline of under 10 s to over 60 s, it is important to note that these iterations still yielded valid physical outputs. The exact underlying cause of their extended runtime is not definitively known. However, such delays could plausibly be caused by temporary I/O bottlenecks, background operating system tasks, or resource contention. Because the physical inputs were strictly identical across these iterations, algorithmic variations such as the solver dynamically reducing its timestep are highly unlikely to be the root cause. This is further supported by the disproportionately low average power draw of these outliers, which suggests the hardware was idling rather

4.2 Study 1: Evaluation of the Time-to-Solution Energy Proxy

than actively performing computations. Despite the increases in execution time between runs, the corresponding energy draw did not appear to scale proportionally, resulting in a degraded linear fit ($R^2 = 0.165$). This suggests that relying just on wall-clock time as an energy metric may be susceptible to external cluster interference. The Z-score filtering methodology was applied to identify and exclude these longer runs from the dataset to isolate the solver's active computational footprint.

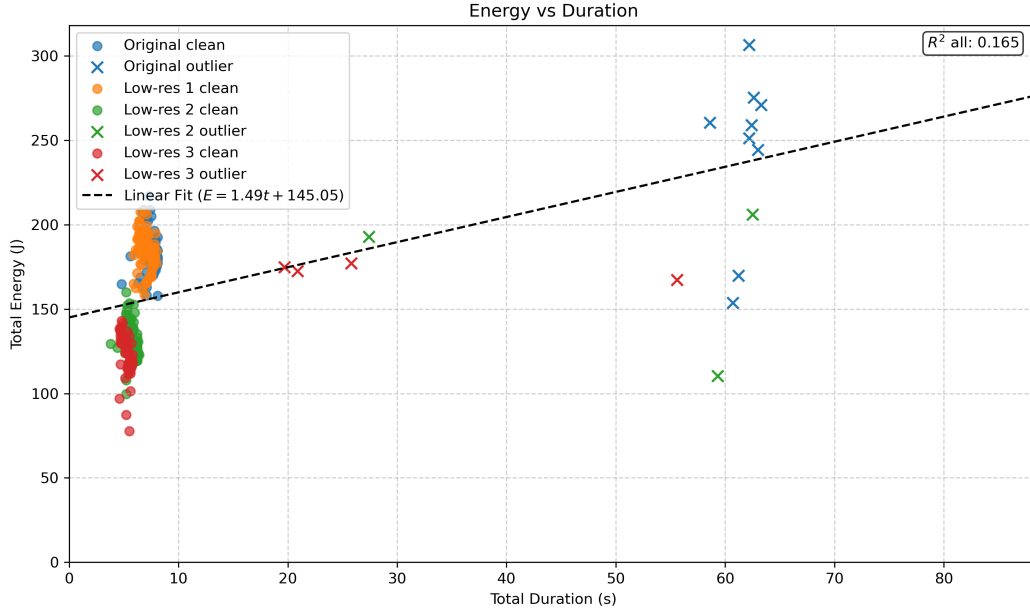


Figure 4.5: Total energy consumption vs. execution duration for all iterations.

After filtering the outliers, the clean dataset was evaluated to test if dynamic power remains constant across iterations. If energy consumption were primarily driven by execution duration, the filtered data would theoretically exhibit a highly linear correlation (near $R^2 = 1.0$). Figure 4.6 reveals that even under clean and stable cluster conditions, this relationship remains surprisingly weak. The linear regression for the clean iterations yields a Coefficient of Determination of $R^2 = 0.518$. A value this low indicates that 48.2% of the variance in energy consumption cannot be explained by execution time alone. This divergence indicates that the solver's hardware usage, and therefore its dynamic power, fluctuates between iterations. Consequently, within the context of this study, time-to-solution appears to be an unreliable standalone proxy for total energy cost.

4 Evaluation and Experimental Results

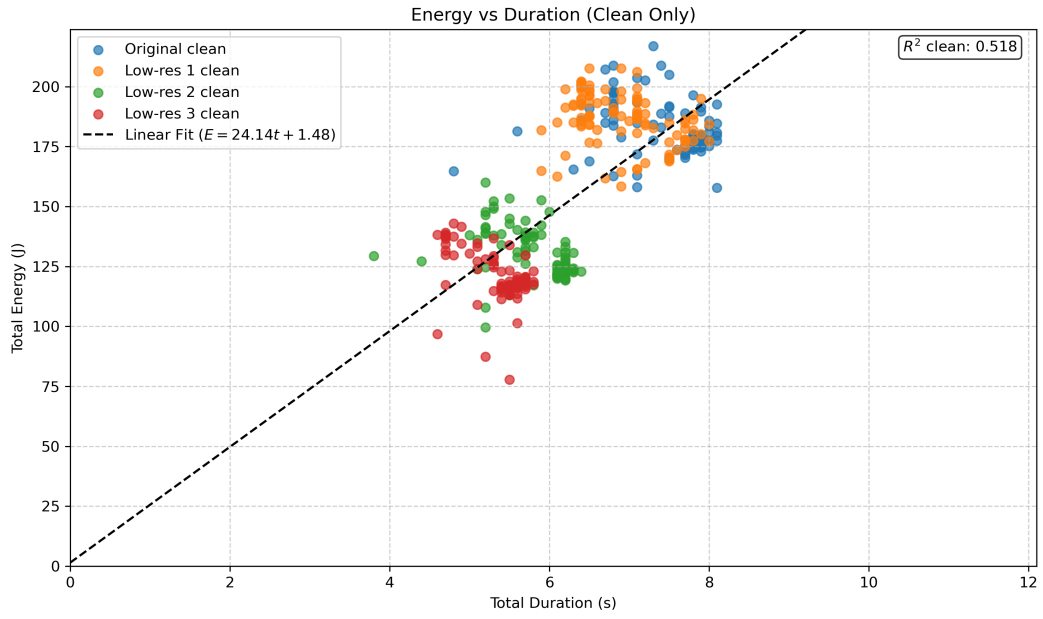


Figure 4.6: Total energy consumption vs. execution duration for all clean iterations.

The underlying cause of this non-linearity is further clarified by examining the average power draw across the different resolution tiers. Figure 4.7 plots the average cumulative energy trajectories for each spatial resolution. The slope of these curves represents the dynamic power draw in Watts. The data illustrates that power is not a static constant; it shifts dynamically based on the grid resolution being processed. For example, the 12.0 m resolution exhibited an average power draw of 22.68 W, while the coarser 15.0 m and 20.0 m resolutions operated at lower intensities of 17.14 W and 16.90 W, respectively.

Resolution Comparison	<i>t</i> -statistic	<i>p</i> -value	Significant ($p < 0.05$)?
10.0 m vs. 12.0 m	-4.684	< 0.001	Yes
10.0 m vs. 15.0 m	12.183	< 0.001	Yes
10.0 m vs. 20.0 m	13.457	< 0.001	Yes
12.0 m vs. 15.0 m	17.084	< 0.001	Yes
12.0 m vs. 20.0 m	18.634	< 0.001	Yes
15.0 m vs. 20.0 m	0.743	0.458	No

Table 4.1: Welch’s *t*-test results comparing the average power draw between different grid resolution ensembles.

4.2 Study 1: Evaluation of the Time-to-Solution Energy Proxy

A Welch’s t-test confirms that these shifts in hardware intensity are statistically significant ($p < 0.001$) across almost all workload transitions. The test also highlights an unexpected dynamic: the 12.0 m resolution draws a significantly higher average power than the more complex 10.0 m baseline ($t = -4.684, p < 0.001$). This suggests that a higher power draw may not strictly indicate a larger total mathematical workload, but could instead reflect higher hardware utilization efficiency. This higher efficiency likely stems from the array dimensions of the 12.0 m grid aligning more favorably with the GPU’s cache capacity or thread block layout. This alignment would decrease memory stalls, allowing the processing cores to operate at a higher, more continuous intensity per second despite having fewer total cells to compute. Conversely, the comparison between the two coarsest grids (15.0 m and 20.0 m) yielded no statistically significant difference in power draw ($t = 0.743, p = 0.458$). This implies that while grid resolution and hardware alignment actively influence dynamic power, the system’s power usage might stabilize once the computational grid becomes sufficiently sparse.

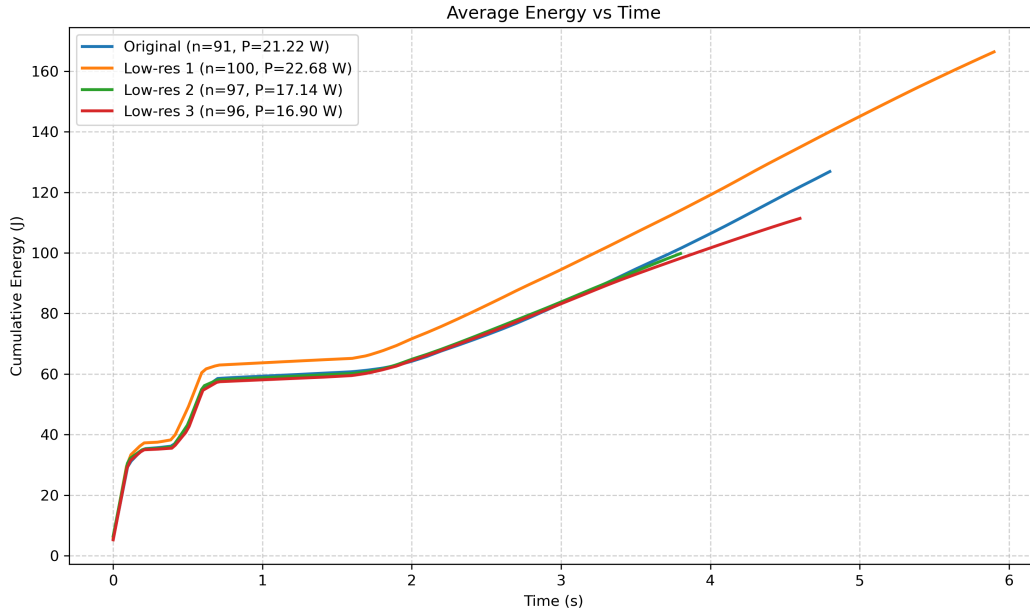


Figure 4.7: Average cumulative energy-time plot of ensembles with different resolutions.

This behavior suggests that altering the spatial resolution of a hydrodynamic simulation does not simply decrease or increase the execution duration. Instead, it can actively alter the intensity of the hardware’s power footprint. Because varying the

computational workload appears to shift the baseline power rate, execution time alone may not reliably predict energy consumption across varying grid scales.

4.3 STUDY 2: PHYSICAL HAZARD UNCERTAINTY QUANTIFICATION

To establish a baseline understanding of how topographic errors affect simulation accuracy, this study investigates the necessity of UQ within our flood case. The primary objective is to quantify how physical input noise propagates through the SynxFlow solver to alter the resulting spatial flood predictions.

4.3.1 EXPERIMENTAL SETUP

Five separate Monte Carlo ensembles were generated, each applying a different level of spatially stationary Gaussian noise to the baseline Digital Elevation Model (DEM). The selected noise standard deviations, $\sigma \in \{0.05, 0.1, 0.2, 0.5, 1.0\}$ m, were chosen to represent the real-world accuracy spectrum of various remote sensing technologies:

- **0.05 m:** Represents optimal drone mapping under ideal conditions, which can achieve absolute uncertainties as low as ± 0.05 m [19].
- **0.1 m:** Represents the industry standard for high-quality, drone-derived terrain maps. This value aligns with official accuracy standards for high-resolution geospatial data, even in lightly vegetated areas [6, 19].
- **0.2 m:** Reflects the loss of accuracy that occurs in taller, dense vegetation, where the true ground is partially obscured from the sensor [19].
- **0.5 m:** Represents severe elevation bias, such as when thick shrubs completely block the ground and are mistakenly recorded as solid terrain by the sensor [19].
- **1.0 m:** Represents older or lower-resolution legacy datasets (such as aerial radar mapping), which typically exhibit vertical errors of 1.0 to 2.0 meters [6].

Each of the five ensembles consists of 100 equiprobable realizations. This sample size was selected as a computationally feasible benchmark to capture primary distributional trends while constraining total execution time and energy expenditure across the multiple noise tiers.

4.3 Study 2: Physical Hazard Uncertainty Quantification

To measure the physical impact of the input noise, the Quantity of Interest (QoI) was defined as the maximum water depth recorded at a specific spatial coordinate [925.0, 605.0].

This specific coordinate was selected because it represents a transitional flood zone. Coordinates situated in the center of a deep riverbed will inevitably flood across all iterations, exhibiting low variance, while coordinates on elevated topography will constantly remain dry. In contrast, a point located on the exact margin of the flood extent is highly sensitive to localized topographical noise. This approach mimics real-world challenges, such as evaluating the localized flood risk for a residential structure built on the absolute edge of a floodplain.

4.3.2 STATISTICAL FRAMEWORK

To quantify the degradation of physical predictions as input data quality drops, three statistical metrics were calculated for the water depth at the QoI across each ensemble:

- **Mean (μ):** Establishes the expected physical water depth (in meters) at the boundary coordinate for each noise tier.
- **Standard Deviation (σ):** Quantifies the absolute physical error introduced into the flow dynamics by the sensor noise.
- **Coefficient of Variation (CV):** Normalizes the absolute error relative to the mean depth ($CV = \frac{\sigma}{\mu}$), providing a dimensionless metric to evaluate relative uncertainty across different perturbation tiers.

4.3.3 RESULTS OF STUDY 2

Table 4.2 summarizes the statistical impact of topographical uncertainty on the simulated water depth at the selected location. Within the context of this specific case study, the data reveals three key observations regarding how input errors propagate through the hydrodynamic solver.

σ_{input} (m)	Mean (m)	Std. Dev (m)	CV (%)
0.05	0.739	± 0.050	6.78%
0.1	0.756	± 0.110	14.50%
0.2	0.774	± 0.206	26.61%
0.5	0.880	± 0.446	50.71%
1.0	1.101	± 0.817	74.21%

Table 4.2: Impact of input uncertainty σ_{input} on the resulting water depth measurements.

As expected, increasing the topographical noise (σ_{input}) directly degrades the certainty of the physical simulation. At high-quality sensor resolutions ($\sigma = 0.05$ m and 0.1 m), the model output remains relatively stable, showing a CV below 15%. However, as the DEM accuracy drops to levels typical of dense vegetation or older legacy datasets ($\sigma \geq 0.5$ m), relative uncertainty increases sharply, with the CV exceeding 50%. At $\sigma = 1.0$ m, the standard deviation of the water depth reaches ± 0.817 m, resulting in a CV of 74.21%. For this specific coordinate, a variance this large suggests that localized flood predictions may become unreliable for risk assessment, as the margin of error approaches the predicted flood depth itself.

While the increase of the standard deviation was anticipated, the behavior of the expected water depth reveals a distinct non-linear effect. Although the injected Gaussian noise was strictly zero-mean ($\mu = 0$), the ensemble mean water depth does not remain constant across ensembles. Instead, the mean water depth rises systematically from 0.739 m at the lowest noise tier to 1.101 m at the highest.

This implies that, in this specific simulation, topographical noise causes more than just random fluctuations. The bumps and dips introduced by the noise likely act as synthetic dams and sinks and these artificial obstructions prevent natural drainage. As a result, the water pools and backs up into the gauge location, leading to the higher localized flood depths observed here.

These initial results highlight why deterministic, single-run hazard assessments can potentially be misleading, especially in locations near riverbeds. Therefore, executing UQ serves as a valuable tool for understanding the true reliability of a flood map.

4.4 STUDY 3: ENERGY CONSUMPTION UNCERTAINTY PROPAGATION

The final study assesses how physical input uncertainty propagates into computational energy variability. This study addresses Sub-question 3, by evaluating whether the topographic noise injected into the digital elevation model inherently alters the total process-attributed energy required to solve the hydrodynamic equations.

4.4.1 EXPERIMENTAL SETUP

To ensure a proper comparison, this study analyzes the exact same five Monte Carlo ensembles ($\sigma \in \{0.05, 0.1, 0.2, 0.5, 1.0\}$ m) generated in [Study 2](#).

Because the baseline simulation parameters remain identical across all runs, any observed change in energy consumption is mathematically isolated. This ensures that variations in power draw are caused by the GPU solver solving altered DEMs.

4.4.2 STATISTICAL FRAMEWORK

To quantify this variation, the resulting energy telemetry for each noise tier is evaluated using three specific statistical metrics:

MEAN (μ): As the standard deviation of the input noise (σ) increases, the simulated terrain becomes artificially "rougher," characterized by synthetic bumps and dips. To remain mathematically stable when water flows over steep or irregular gradients, hydrodynamic solvers like SynxFlow must rely on adaptive timestepping. Rougher terrain forces the solver to take smaller timesteps to solve the flow.

STANDARD DEVIATION (STD. DEV.): The standard deviation measures the spread of the energy in Joules. This metric quantifies how much the power draw swings from one individual iteration to the next, even when the overall noise distribution (σ) remains the same.

90% CONFIDENCE INTERVAL (5TH AND 95TH PERCENTILES): This metric is selected to quantify the dispersion of total energy consumption across ensemble realizations and capture upper-bound computational demands. Evaluating the spread between the 5th and 95th percentiles provides a measure of resource variability while remaining robust against extreme statistical outliers.

4.4.3 RESULTS OF STUDY 3

The initial evaluation of energy uncertainty (Run 1) analyzed a 100-iteration Monte Carlo ensemble for each noise tier. As shown in Figure 4.8 and summarized in Table 4.3, the findings from this first execution window suggested that as topographic noise (σ) increased, both the average energy consumption and the run-to-run variance expanded. For example, the mean energy rose from 184.78 J at $\sigma = 0.05$ m to 212.80 J at $\sigma = 1.00$ m, while the standard deviation nearly tripled from ± 19.71 J to ± 55.75 J.

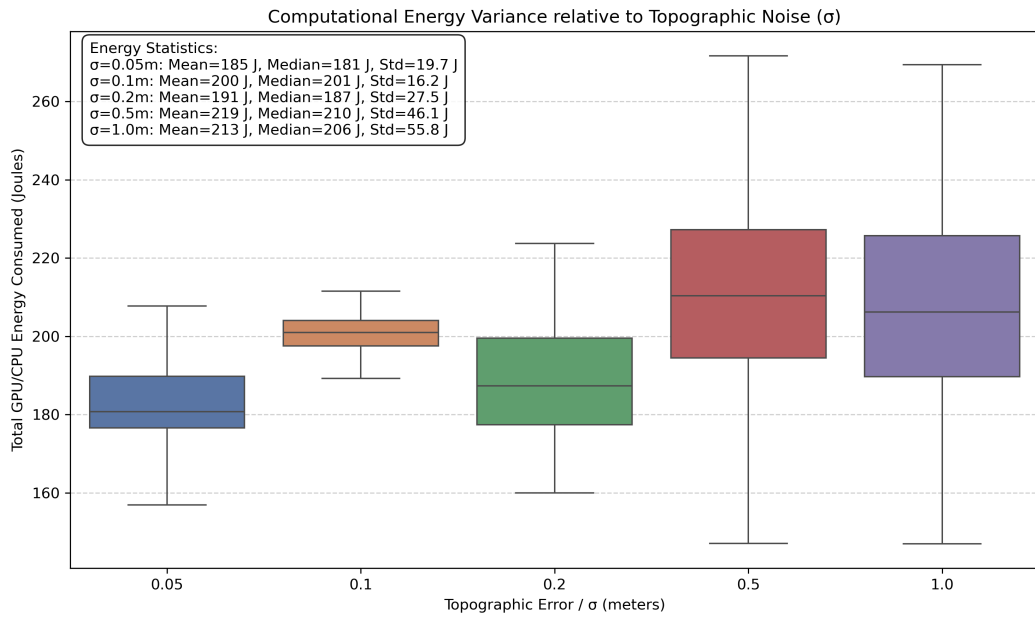


Figure 4.8: Boxplots of total energy consumption for each topographic noise level for run 1

Topographic Noise (σ)	Mean (μ)	Median	Std. Dev.	90% Confidence Interval
0.05 m	184.78 J	180.73 J	± 19.71 J	[169.25 J, 207.33 J]
0.10 m	199.81 J	200.97 J	± 16.24 J	[176.86 J, 211.62 J]
0.20 m	191.38 J	187.31 J	± 27.48 J	[167.91 J, 220.32 J]
0.50 m	219.28 J	210.33 J	± 46.09 J	[152.31 J, 319.68 J]
1.00 m	212.80 J	206.24 J	± 55.75 J	[147.81 J, 293.47 J]

Table 4.3: Statistical summary of total computational energy consumption for Run 1 (100 iterations).

4.4 Study 3: Energy Consumption Uncertainty Propagation

However, as demonstrated in [Study 1](#), energy metrics in a shared HPC environment can be susceptible to background traffic. To verify whether the energy profile observed in Run 1 was an inherent property of the solver or just a reflection of the cluster’s state during that specific execution window, the exact same experimental setup was duplicated in a separate, independent cluster allocation (Run 2).

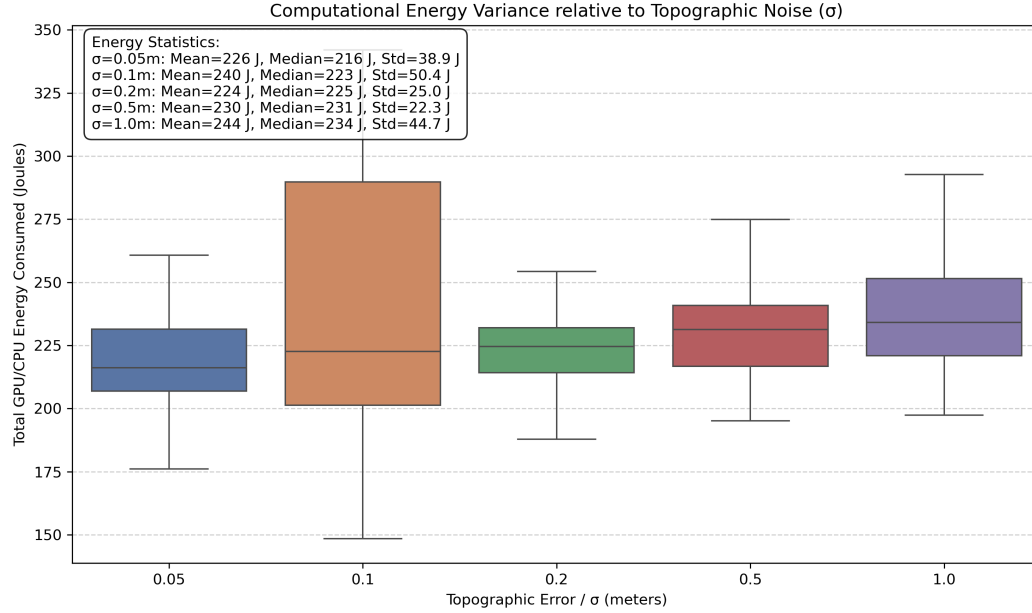


Figure 4.9: Boxplots of total energy consumption for each topographic noise level for run 2

Topographic Noise (σ)	Mean (μ)	Median	Std. Dev.	90% Confidence Interval
0.05 m	226.40 J	216.10 J	± 38.88 J	[183.00 J, 319.78 J]
0.10 m	239.76 J	222.58 J	± 50.42 J	[182.70 J, 332.58 J]
0.20 m	224.47 J	224.51 J	± 24.96 J	[189.88 J, 255.58 J]
0.50 m	230.38 J	231.25 J	± 22.30 J	[202.45 J, 259.73 J]
1.00 m	244.44 J	234.16 J	± 44.74 J	[210.16 J, 331.28 J]

Table 4.4: Statistical summary of total computational energy consumption for Run 2 (100 iterations).

The telemetry from Run 2, detailed in [Figure 4.9](#) and [Table 4.4](#), also signals the impact of hardware volatility. While Run 2 exhibited similar internal trends regarding the expansion of variance, its overall baseline energy footprint was consistently higher across all tiers. For instance, at the $\sigma = 0.05$ m tier, the mean energy jumped from 184.78 J in Run 1 to 226.40 J in Run 2. Furthermore, Run 2 contains a striking

4 Evaluation and Experimental Results

anomaly at the $\sigma = 0.10$ m noise level. Despite representing a high-quality terrain input, this tier recorded a disproportionately high mean of 240J and the largest standard deviation of the run at ± 50.4 J. Because [Study 1](#) established that idle cluster stalls reduce dynamic power rates rather than surging computational energy, this elevated energy footprint is unlikely to stem from pure hardware idling. Instead, it suggests that specific DEMs within this tier may have triggered aggressive adaptive timestepping in the solver. These discrepancies illustrate that baseline energy profiles can fluctuate significantly between execution windows due to a combination of input-uncertainty and cluster conditions.

Because evaluating a single execution window might lead to biased conclusions regarding total operational cost, the telemetry from both runs was merged into a single 200-iteration ensemble per noise tier. This combined dataset aims to mitigate cluster-induced variance and establish a more generalized understanding of the solver’s energy behavior. Figure 4.10 and Table 4.5 summarize the statistical findings for this merged dataset.

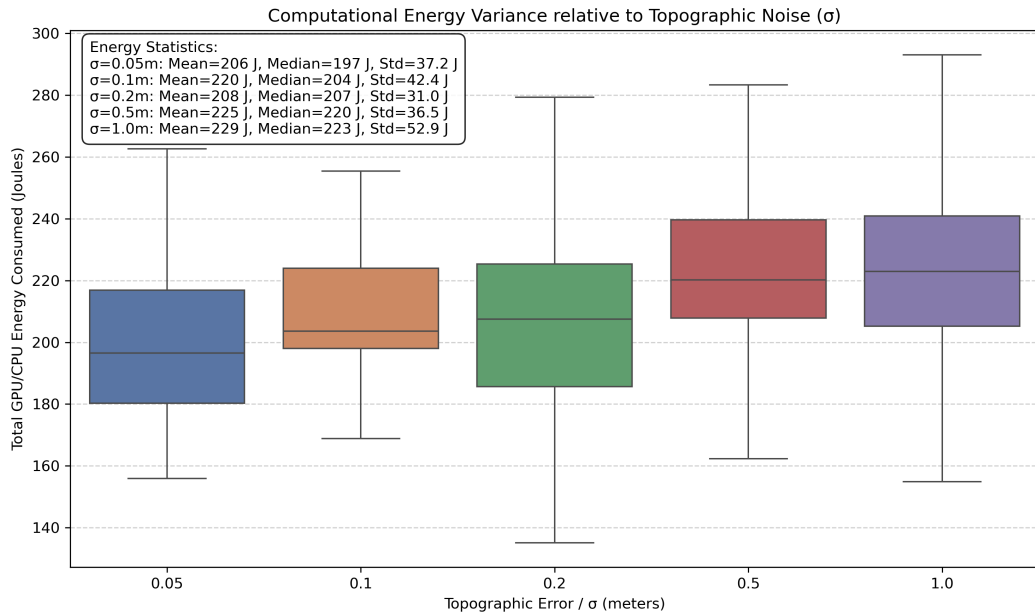


Figure 4.10: Boxplots of total energy consumption for combined results of runs 1 and 2

The combined telemetry data offers three key observations regarding how topographic uncertainty might propagate into computational energy metrics within the scope of this specific study.

Topographic Noise (σ)	Mean (μ)	Median	Std. Dev.	90% Confidence Interval
0.05 m	205.59 J	196.53 J	± 37.15 J	[171.09 J, 301.03 J]
0.10 m	219.79 J	203.66 J	± 42.39 J	[181.82 J, 318.61 J]
0.20 m	207.92 J	207.50 J	± 31.00 J	[172.34 J, 249.60 J]
0.50 m	224.83 J	220.15 J	± 36.54 J	[170.81 J, 299.63 J]
1.00 m	228.62 J	222.91 J	± 52.85 J	[167.94 J, 312.16 J]

Table 4.5: Statistical summary of total computational energy consumption for the combined 200-iteration ensembles.

By comparing the median and mean, the data reveals a distinction between the solver’s typical operational cost and its upper-bound tail behavior. Across all ensembles, the mean consistently exceeds the median. This right-skewed distribution indicates that while the baseline energy draw remains stable under typical realizations, steep localized gradients trigger adaptive timestepping, producing a tail of higher-energy runs that elevates the mean. As input noise increases, the overall energy cost trends upward due to these increased numerical stability constraints.

Beyond altering the average energy cost, higher topographic noise appears to degrade the predictability of the solver’s energy footprint. The spread of energy consumption generally increases as the input noise increases. For the 0.05 m and 0.20 m tiers, the standard deviations are 37.15 J and 31.00 J, respectively. However, at the 1.00 m noise tier, the standard deviation expands to 52.85 J. This indicates that when a digital elevation model is highly uncertain, the computational energy required to process it might also become highly uncertain, leading to more run-to-run variance even within the exact same noise distribution.

The expansion of the 90% confidence interval highlights a potential challenge for energy estimation in HPC environments. The combined data shows that at the 1.00 m noise tier, the 95th percentile upper bound reaches 312.16 J, which is nearly 84 J higher than the mean. Within the context of this study, this suggests that relying on a single deterministic profile to predict the maximum energy budget for a full stochastic ensemble might result in under-allocation. Consequently, safety margins for computational budgets may need to be scaled dynamically based on the quality of the input data, with poorer input data potentially requiring a wider safety net.

5 SUMMARY AND FUTURE WORK

This report addressed the gap between geohazard uncertainty quantification and computational energy profiling. By coupling the SynxFlow hydrodynamic solver with the Alumet telemetry framework, this work transitioned from relying on execution time as a proxy to directly measuring process-isolated energy consumption for stochastic flood simulations. This chapter summarizes the key findings, acknowledges experimental limitations, and outlines future research directions.

5.1 SUMMARY

This research investigated how input data uncertainty and execution environment volatility impact the energy predictability of GPU-accelerated flood solvers. The core research questions from Section 1.2 were addressed as follows:

- RQ1* **Methodologies for process-level energy attribution:** A custom Python framework was developed to integrate SynxFlow with Alumet. By utilizing dynamic Process ID tracking and scaling hardware power by utilization percentages, this methodology isolated the solver’s true energy footprint from shared cluster noise.
- RQ2* **Execution time as an energy proxy:** Preliminary results indicated that execution time may be an unreliable standalone proxy for computational energy in this context. Welch’s t-tests suggested that altering the computational grid’s spatial resolution can shift the hardware’s dynamic power intensity. Time-to-solution might not fully capture this energy variability, as power rates appeared to fluctuate based on grid-to-GPU alignment.
- RQ3* **Uncertainty propagation to energy variability:** Initial observations suggested that as spatial input uncertainty (σ) increases, both the average computational energy and run-to-run variance show a tendency to expand. Rougher terrain may trigger adaptive timestepping, potentially producing right-skewed energy distributions. This indicates that highly uncertain DEMs could lead to

highly uncertain energy footprints, suggesting that single deterministic profiles might underestimate the energy requirements of stochastic ensembles.

5.2 LIMITATIONS

While establishing a foundational link between input uncertainty and energy consumption, this study has several boundaries. First, the simplified error modeling relies on a zero-mean Gaussian noise model ($\epsilon \sim \mathcal{N}(0, \sigma^2)$), which assumes isotropic errors and oversimplifies the spatially correlated and biased errors of real-world remote sensing. Second, the scope of the scenario is limited, as the findings stem from a single flood simulation. Consequently, the observed non-linear hydrodynamic effects may not uniformly apply to varying catchments or urban floodplains.

Additionally, due to computational expense, the ensemble sizes were restricted to 100 iterations. While this sample size captures primary trends, it may be insufficient to fully stabilize extreme tail behaviors within the energy distributions. Finally, the observed power shifts are inherently tied to the hardware specificity of the Gaia compute cluster and its NVIDIA GPUs, meaning these exact energy dynamics may vary across different computing architectures.

5.3 FUTURE WORK

The findings of this thesis open several promising directions for further research. First, future studies must improve the error model by incorporating spatially correlated noise alongside a negative bias shift. Because spaceborne sensors frequently overestimate ground heights by capturing vegetation and buildings instead of bare earth [14], an accurate error model must simulate the removal of this positive sensor bias by lowering the elevation values in obstructed areas.

Furthermore, the topographic scope of the framework should be expanded. Testing the methodology on diverse flood cases and different DEMs will help determine if specific terrain features inherently demand higher adaptive energy costs. Finally, scaling the Monte Carlo simulations to larger ensembles is necessary to observe if the variability in energy consumption fully stabilizes. Capturing this stabilization is critical for establishing reliable, carbon-aware HPC scheduling limits.

BIBLIOGRAPHY

1. J. Aaron. "ORIN-3D – A new model for efficient simulation of landslide motion on a GPU using CUDA". *Computers and Geotechnics* 153, 2023, p. 105078. doi: [10.1016/j.compgeo.2022.105078](https://doi.org/10.1016/j.compgeo.2022.105078). URL: <https://doi.org/10.1016/j.compgeo.2022.105078>.
2. P. Abbaszadeh, D. F. Muñoz, H. Moftakhari, K. Jafarzadegan, and H. Moradkhani. "Perspective on uncertainty quantification and reduction in compound flood modeling and forecasting". *iScience* 25:10, 2022, p. 105201. doi: [10.1016/j.isci.2022.105201](https://doi.org/10.1016/j.isci.2022.105201). URL: <https://doi.org/10.1016/j.isci.2022.105201>.
3. M. Abily, N. Bertrand, O. Delestre, P. Gourbesville, and C.-M. Duluc. "Spatial global sensitivity analysis of high resolution classified topographic data use in 2D urban flood modelling". *Environmental Modelling & Software* 77, 2016, pp. 183–195. doi: [10.1016/j.envsoft.2015.12.002](https://doi.org/10.1016/j.envsoft.2015.12.002).
4. E. Ah-Woane, T. Davarend, F. Cordier, Z. Amama, B. Basic, M. Paipuri, A. Sochinskii, M. Assaba, S. Majdalani, R. Moussa, M. Abily, and O. Delestre. "Toward Brague river flood modelling 2: efforts to optimize HPC resources for a TELEMAT-2D code based on shallow water equations". *Digital Water* 3:1, 2025, pp. 1–27. doi: [10.1080/28375807.2025.2474241](https://doi.org/10.1080/28375807.2025.2474241). URL: <https://doi.org/10.1080/28375807.2025.2474241>.
5. Alumet Developers. *Alumet: A modular and extensible performance and energy profiling framework*. <https://github.com/alumet-dev/alumet>. Accessed: 2026-07-18. 2026.
6. ASPRS Map Accuracy Standards Working Group. "ASPRS Positional Accuracy Standards for Digital Geospatial Data". *Photogrammetric Engineering & Remote Sensing* 81:3, 2015, A1–A26. doi: [10.14358/PERS.81.3.A1-A26](https://doi.org/10.14358/PERS.81.3.A1-A26). URL: <https://doi.org/10.14358/PERS.81.3.A1-A26>.

7. M. Büttner, C. Alt, T. Kenter, C. Plessl, and V. Aizinger. “Analyzing performance portability for a SYCL implementation of the 2D shallow water equations”. *The Journal of Supercomputing* 81:12, 2025, p. 772. DOI: [10.1007/s11227-025-07063-7](https://doi.org/10.1007/s11227-025-07063-7). URL: <https://doi.org/10.1007/s11227-025-07063-7>.
8. A. Correa and G. Chang. *alumet-insight: A tool to gain insight from Alumet CSV output*. <https://github.com/thealanjason/alumet-insight>. Accessed: 2026-07-23. 2026.
9. E. Edgü and A. Correa. *topographic_uq_energy: Soft Sensors to Measure Energy Consumption of Topographic Uncertainty Propagation in Mass Flow Simulations*. https://github.com/thealanjason/topographic_uq_energy. GitHub repository. 2026.
10. N. Fraehr, Q. J. Wang, W. Wu, and R. Nathan. “Development of a fast and accurate hybrid model for floodplain inundation simulations”. *Water Resources Research* 59:6, 2023, e2022WR033836. DOI: [10.1029/2022WR033836](https://doi.org/10.1029/2022WR033836). URL: <https://doi.org/10.1029/2022WR033836>.
11. H. H. Holm, A. R. Brodtkorb, and M. L. Sætra. “GPU Computing with Python: Performance, Energy Efficiency and Usability”. *Computation* 8:1, 2020, p. 4. DOI: [10.3390/computation8010004](https://doi.org/10.3390/computation8010004). URL: <https://doi.org/10.3390/computation8010004>.
12. N. M. Hunter, P. D. Bates, S. Neelz, G. Pender, I. Villanueva, C. N. Peacock, D. Liang, R. A. Falconer, A. Bates, M. S. Horritt, N. G. Wright, A. Scott, M. G. Wilson, and F. Alcrudo. “Benchmarking 2D hydraulic models for urban flooding”. *Proceedings of the Institution of Civil Engineers - Water Management* 161:1, 2008, pp. 13–30. DOI: [10.1680/wama.2008.161.1.13](https://doi.org/10.1680/wama.2008.161.1.13).
13. S. Kang, J. Yin, and D. Yamazaki. “CaMa-Flood-GPU: a GPU-based hydrodynamic model implementation for scalable global simulations”. *Geoscientific Model Development* 19, 2026, pp. 5623–5640. DOI: [10.5194/gmd-19-5623-2026](https://doi.org/10.5194/gmd-19-5623-2026). URL: <https://doi.org/10.5194/gmd-19-5623-2026>.
14. M. Meadows, S. Jones, and K. Reinke. “Vertical accuracy assessment of freely available global DEMs (FABDEM, Copernicus DEM, NASADEM, AW3D30 and SRTM) in flood-prone environments”. *International Journal of Digital Earth* 17:1, 2024, p. 2308734. DOI: [10.1080/17538947.2024.2308734](https://doi.org/10.1080/17538947.2024.2308734). URL: <https://doi.org/10.1080/17538947.2024.2308734>.

15. J. Oksanen. "Digital elevation model error in terrain analysis". PhD thesis. Helsinki, Finland: University of Helsinki, 2006. ISBN: 952-10-3350-9. URL: <http://urn.fi/URN:ISBN:952-10-3350-9>.
16. C. Sánchez-Linares, M. de la Asunción, M. J. Castro, T. Morales-de Luna, and J. Macías. "Uncertainty quantification in tsunami modeling using multi-level Monte Carlo finite volume method". *Journal of Mathematics in Industry* 6:1, 2016, p. 5. DOI: [10.1186/s13362-016-0022-8](https://doi.org/10.1186/s13362-016-0022-8).
17. J. T. S. Savage, P. Bates, J. Freer, J. Neal, and G. Aronica. "When does spatial resolution become spurious in probabilistic flood inundation predictions?" *Hydrological Processes* 30:13, 2016, pp. 2014–2032. DOI: [10.1002/hyp.10749](https://doi.org/10.1002/hyp.10749).
18. J. Shaw, G. Kesserwani, J. Neal, P. Bates, and M. K. Sharifian. "LISFLOOD-FP 8.0: the new discontinuous Galerkin shallow-water solver for multi-core CPUs and GPUs". *Geoscientific Model Development* 14:6, 2021, pp. 3577–3602. DOI: [10.5194/gmd-14-3577-2021](https://doi.org/10.5194/gmd-14-3577-2021). URL: <https://doi.org/10.5194/gmd-14-3577-2021>.
19. J. Van der Sluijs, R. H. Fraser, and T. C. Lantz. "Replicability of Digital Terrain Models and Canopy Height Models Derived from Drone Photogrammetry". *Remote Sensing* 18:4, 2026, p. 627. DOI: [10.3390/rs18040627](https://doi.org/10.3390/rs18040627). URL: <https://doi.org/10.3390/rs18040627>.
20. N. Stojanovic and D. Stojanovic. "Parallelizing Multiple Flow Accumulation Algorithm using CUDA and OpenACC". *ISPRS International Journal of Geo-Information* 8:9, 2019, p. 386. DOI: [10.3390/ijgi8090386](https://doi.org/10.3390/ijgi8090386). URL: <https://doi.org/10.3390/ijgi8090386>.
21. Y. Tian, C. Lin, and K. Li. "Managing performance and power consumption tradeoff for multiple heterogeneous servers in cloud computing". *Cluster Computing* 17:3, 2014, pp. 943–955. DOI: [10.1007/s10586-013-0326-z](https://doi.org/10.1007/s10586-013-0326-z). URL: <https://doi.org/10.1007/s10586-013-0326-z>.
22. X. Xia, Q. Liang, X. G. Ming, and J. Hou. "An efficient and stable hydrodynamic model with novel source term discretization schemes for overland flow and flood simulations". *Water Resources Research* 53:5, 2017, pp. 3730–3759. DOI: [10.1002/2016WR020055](https://doi.org/10.1002/2016WR020055). URL: <https://doi.org/10.1002/2016WR020055>.
23. X. Xia and X. Ming. "SynxFlow: A GPU-accelerated Python package for multi-hazard simulations". *Journal of Open Source Software* 11:118, 2026, p. 7586. DOI: [10.1002/2016WR020055](https://doi.org/10.1002/2016WR020055).

Bibliography

- 21105/joss.07586. URL: <https://doi.org/10.21105/joss.07586>.
24. X. Xia and X. Ming. *SynxFlow Documentation*. <https://synxflow.readthedocs.io/en/latest/>. Accessed: 2026-07-19. 2026.
25. H. Zhao and J. Kowalski. "Topographic uncertainty quantification for flow-like landslide models via stochastic simulations". *Natural Hazards and Earth System Sciences* 20:5, 2020, pp. 1441–1461. DOI: [10.5194/nhess-20-1441-2020](https://doi.org/10.5194/nhess-20-1441-2020). URL: <https://doi.org/10.5194/nhess-20-1441-2020>.